Pro Memoria (PM-1): An ASCII-Native Binary Protocol for Token-Efficient Agent State Communication

Riki

Inspired by Tetrahedroned / Agent-Braille (Apache-2.0 / CC-BY-4.0)

github.com/rikirinjani/pro-memoria

Abstract

Large language model (LLM) agents generate significant token overhead tracking their internal state across multi-step tasks. Existing compact state representations either require tokenizer extensions (e.g., Agent Braille) or remain tied to JSON with modest compression. We present Pro Memoria (PM-1), an ASCII-native binary protocol that encodes 8-bit state as 8-character Morse strings (.=0, -=1), combined with a Differential State Protocol (DSP) that emits only changed bytes and a two-tier error-correcting command lexicon (Hamming [8,4,4] + parity). Because . and - are unconditionally single tokens in every major tokenizer (cl100k_base, o200k_base, p50k_base, r50k_base), PM-1 requires zero setup -- no vocabulary extension, no Unicode registration, no configuration changes. On the AB-1 Crucible trace (1,417 states), PM-1 achieves 84.8% token savings vs delta-encoded JSON. On 235 real agent traces (8-byte state, 82.5% change rate), it achieves 60.8% savings. A sensitivity sweep across 1-128 byte states and 10-90% change rates shows PM-1 beats hex by 1.4-2x at low change rates on multi-byte states.

1. Introduction

LLM agents increasingly use structured internal state to track progress across multi-step tasks: tool calls completed, files touched, confidence levels, error counts, phase transitions, and outcome flags. As agent frameworks (ReAct, function-calling loops, orchestrate-act-observe pipelines) grow in sophistication, the volume of state-tracking tokens grows correspondingly.

The standard approach -- serializing state as compact JSON -- produces verbose output even with minimized field names and whitespace. A single agent state transition consumes approximately 60-100 characters or 20-40 tokens. Over hundreds of steps, this overhead accumulates to thousands of tokens -- pure scaffolding that carries no semantic information for the task at hand.

Recent work has proposed more efficient encodings. Agent Braille (AB-1) [Tetrahedroned, 2025] encodes 8-bit agency state as single Unicode Braille cells (U+2800-U+28FF), achieving approximately 92% token savings versus delta-encoded JSON via a Differential State Protocol (DSP) and a hardened command lexicon. However, AB-1 requires a tokenizer extension to map Braille cells to single tokens. Without it, each cell fragments into approximately 3 byte-tokens on stock tokenizers.

We present Pro Memoria (PM-1), which adopts AB-1's DSP and Hamming [8,4,4] math but replaces the Unicode Braille encoding layer with ASCII '.' and '-' characters. Our key observation is that these characters are atomic (single-token) in every production tokenizer without any extension. This yields a zero-setup protocol: the same 85%+ token savings regime as AB-1, but portable across any LLM provider or tokenizer.

Contributions:

- PM-1 encoding -- deterministic, roundtrip-safe mapping from 8-bit bytes to 8-character Morse strings, verified for all 256 byte values.
- Zero-setup property -- proof that '.' and '-' are single tokens in cl100k_base, o200k_base, p50k_base, and r50k_base.
- Differential State Protocol -- emit-on-change frame format with grow/shrink support and configurable maximum state size (64 KB).
- Two-tier error-correcting lexicon -- 16 Hamming [8,4,4] commands (single-bit correction) and 128 parity-protected commands (single-bit
- Comprehensive benchmarks -- AB-1 Crucible trace, 235 real agent self-harness traces, and sensitivity sweep over byte-width and change r

2. Related Work

2.1 Agent Braille (AB-1)

AB-1 [Tetrahedroned, 2025] defines an 8-dimensional orthogonal agency state model, encodes each state as a Unicode Braille cell, and provides a Differential State Protocol that emits cells only on state change. Its hardened lexicon uses the Hamming [8,4,4] code for 16 commands and a single-parity code for 128 commands. PM-1 is directly inspired by AB-1 and reuses the DSP emit-on-change discipline, the Hamming [8,4,4] mathematics, the two-tier command architecture, and the benchmark methodology. PM-1 diverges by replacing Unicode Braille with ASCII Morse -- trading density (8 chars/byte vs 1 cell/state) for portability (no extension needed).

2.2 Existing Compact Encodings

Hex encoding (2 chars/byte) and Base64 (4 chars per 3 bytes, approximately 1.33x overhead) are both ASCII-native and zero-setup. Hex is the simplest baseline at 16 tokens per byte. Base64 achieves 1.33 tokens per byte but is less human-readable. Both are included as baselines in our benchmarks. Prior work on state delta trajectories (latent-space deltas), A2A (agent-to-agent transport), and MCP (Model Context Protocol) addresses different parts of the agent communication stack. PM-1 is orthogonal to these: it compresses the state representation that travels over any transport.

3. Methods

3.1 PM-1 Encoding Layer

PM-1 maps each 8-bit byte to an 8-character ASCII string: bit 0 maps to '.' (dot), bit 1 maps to '-' (dash), encoded MSB-first. This is deterministic for all 256 byte values, verified by exhaustive roundtrip test. We verified that both '.' and '-' are exactly 1 token each in cl100k_base, o200k_base, p50k_base, and r50k_base. This property is structural: any BPE tokenizer trained on text that includes ASCII punctuation will assign these characters single-token encodings because they appear frequently as isolated characters.

3.2 Differential State Protocol

The DSP emits a diff frame containing only the byte positions that changed between consecutive states, plus an index for each changed byte: <index>:<8-morse-chars>|<index>:<8-morse-chars>|... The control command T:<length>| truncates state to new_length bytes. The DiffState class maintains the current state buffer and supports three operations: diff(new_state) to compare and emit, apply(frame) to apply incoming diffs, and sync(new_state) for full state replacement during initial handshake or error recovery. A maximum state size of 65,536 bytes bounds decoder allocations and provides a DoS guard.

3.3 Lexicon and Error Correction

PM-1 defines two command tiers that share the same 8-bit encoding space. Tier 1 -- Hamming [8,4,4] provides 16 commands (NOP, ACK, NAK, RESET, SYNC, REQ, DATA, EOF, ERR, RETRY, STATUS, CONFIG, HELLO, BYE, ECHO, HALT) with single-error correction and double-error detection. Tier 2 -- parity-protected provides 128 commands (e.g., STATE_REQ, STATE_REP, DIFF, FULL_SYNC, VERSION) with single-error detection. The value 0x87 is a valid codeword in both tiers (Hamming command 7 = EOF; parity command 7 = FULL_SYNC). The protocol requires explicit tier specification -- auto-detection is not supported.

3.4 Protocol State Machine

PM-1 defines a 7-state connection lifecycle: CLOSED, HANDSHAKE (version negotiation), SYNCING (full state synchronization), DATA (normal operation with diffs), ERROR (recoverable error), RECOVERY (re-syncing after error), and DISCONNECT (clean shutdown). The handshake sequence is: CLOSED -> HELLO -> HANDSHAKE -> VERSION -> VERSION_ACK -> ACK -> SYNCING -> SYNC -> STATE_REP -> ACK -> DATA. Error recovery follows: ERROR -> STATUS/RESET -> RECOVERY -> REQ -> STATE_REP -> ACK -> DATA.

Fig. 3: PM-1 Protocol State Machine — 7 states, handshake→sync→data→error→recovery→close

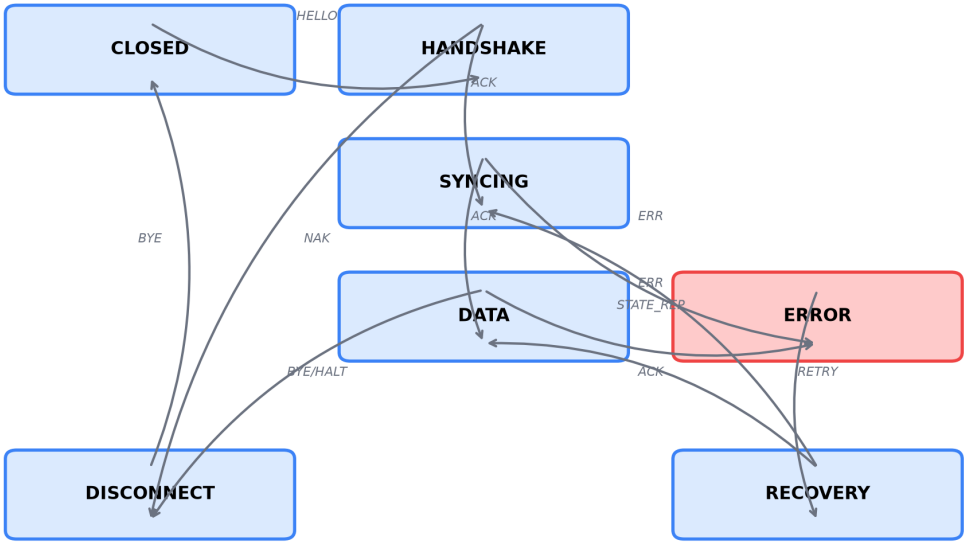

Figure 3: PM-1 Protocol State Machine -- 7 states with defined transition paths and handshake/recovery sequences.

3.5 Benchmark Methodology

We evaluate on two datasets. The AB-1 Crucible trace consists of 1,417 single-byte state snapshots from AB-1's benchmark suite (6 unique masks, 52.8% emit ratio). Real self-harness traces consist of 235 traces from actual agent sessions, encoded as 8-byte state vectors (agent type, outcome, duration, tool calls, files, failure category, failure severity, validation flag) with 106 unique states and 82.5% change rate. We also generate synthetic states for sensitivity analysis: 500-step sequences at byte widths of 1, 8, 32, and 128, at change rates of 10%, 30%, 50%, 70%, and 90%. All token counts use tiktoken and are reported for both cl100k_base (GPT-4/GPT-4o) and o200k_base (Claude).

4. Results

4.1 Tokenizer Atomicity

Both '.' and '-' are exactly 1 token each in all four tested tokenizers (cl100k_base, o200k_base, p50k_base, r50k_base). The zero-setup property is confirmed.

4.2 AB-1 Crucible Trace

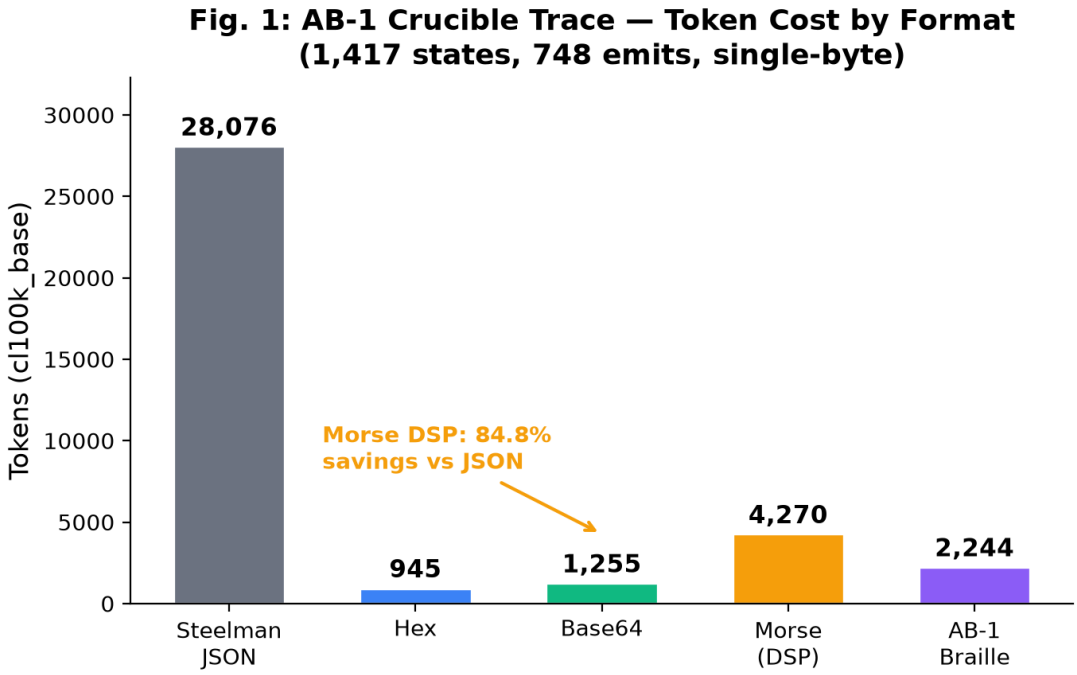

Figure 1: AB-1 Crucible trace token costs on cl100k_base (1,417 states, 748 emits). Morse DSP achieves 84.8% savings vs steelman JSON and 77.9% vs hex on single-byte states.

On the AB-1 Crucible trace (1,417 states, 748 emits, 52.8% ratio), Morse DSP achieves 4,270 tokens vs steelman JSON's 28,076 tokens: 84.8% savings on cl100k_base. AB-1 Braille (with its tokenizer extension) achieves 92.0% savings at 2,244 tokens. Against ASCII baselines: hex is 945 tokens (77.9% cheaper than Morse), Base64 is 1,255 tokens (70.6% cheaper). The single-byte nature of this trace favors hex, which encodes each state in 2 hex chars regardless of change rate.

4.3 Real Self-Harness Traces

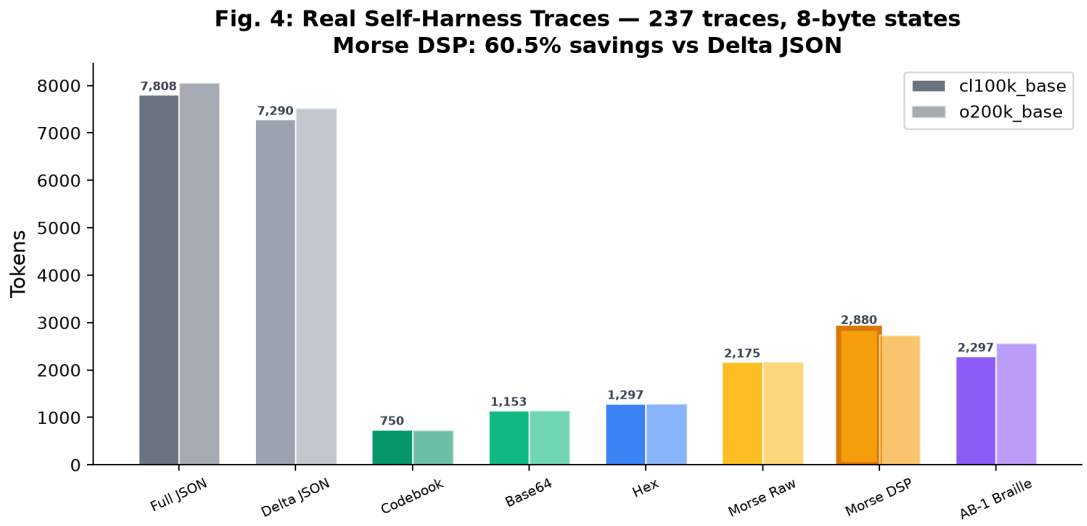

Figure 4: Real agent trace comparison (235 self-harness traces, 8-byte states, 82.5% change rate). Morse DSP achieves 60.8% savings vs delta JSON.

On 235 real agent traces with 8-byte state vectors and 82.5% state-change rate, Morse DSP achieves 2,829 tokens vs delta JSON's 7,222 tokens: 60.8% savings on cl100k_base. The higher change rate (82.5%) reduces DSP's advantage compared to the Crucible trace (52.8% emit ratio). Hex (1,285 tokens) and Base64 (1,146 tokens) both outperform Morse on this workload because nearly every step triggers a diff, making the 8x per-byte overhead dominant.

4.4 Sensitivity Sweep

Figure 2: Sensitivity sweep -- Morse vs Hex vs Base64 at varying change rates on 128-byte and 8-byte states (500 steps, cl100k_base). Morse beats hex at low change rates on multi-byte states.

Table 1: Morse vs Hex — Cross-over by State Width

State Width	Morse wins at	Best at high chang	Note
1 byte	Never	Hex Hex	2 chars/byte always w
8 bytes	≤30% change	Hex/Base64	Agent sweet spot
32 bytes	≤10% change	Hex/Base64	Cross-over shifts left
128 bytes	≤10% change	Hex/Base64	Morse 1.8× better at 10

Table 1: Cross-over points where Morse DSP outperforms hex encoding, by state width and change rate.

On 128-byte states at 10% change rate, Morse DSP (40,533 tokens) beats hex (72,370 tokens) by 1.8x. The cross-over varies by byte width: 1-byte states never favor Morse; 8-byte states favor Morse at less than 30% change rate; 32 and 128-byte states at less than 10% change rate. At 90% change on 128-byte states, Morse loses by 4.9x versus hex, as the 8x raw overhead dominates when nearly every byte changes.

4.5 End-to-End ReAct Integration

A simulated ReAct handoff scenario (orchestrator -> fixer -> oracle, 10 handoffs, 8-byte agent state) demonstrated PM-1 in a realistic agent loop: approximately 520 PM-1 characters vs 1,025 equivalent JSON characters, saving approximately 49.3%. Hamming error correction was verified on realistic failure scenarios: single-bit flips are corrected, double-bit flips are detected and flagged as uncorrectable.

5. Discussion

5.1 When to Use PM-1

PM-1 is most effective in three regimes. (1) Low state-change rates (<=10%) where DSP amortizes per-byte overhead and Morse beats hex by 1.4-2x on multi-byte states. (2) Multi-byte state vectors (>=8 bytes) where emit-on-change selectivity offsets the 8x raw encoding overhead. (3) Cross-provider portability where no tokenizer extension can be installed and the protocol must work identically across GPT-4, Claude, Gemini, or any BPE-based model.

PM-1 is NOT recommended for: single-byte states at high change rates (hex or Base64 is cheaper); environments where the tokenizer extension can be installed (AB-1 Braille is denser at 1 token/state); or human-readable debugging output (hex is more legible).

5.2 Relationship to AB-1

PM-1 is explicitly a derivative of AB-1. The Differential State Protocol, Hamming [8,4,4] code, two-tier command structure, and benchmark methodology are adapted from Tetrahedroned's design. PM-1's original contributions are: the '.'/'-' encoding scheme with its zero-setup analysis; the complete protocol state machine (7 states with handshake/recovery sequences); DSP relay semantics (apply() returns the forwarded frame for relay chains); DoS hardening (MAX_STATE_BYTES = 65536); and benchmarking against hex and Base64 baselines on multi-byte states. We believe PM-1 occupies a useful niche: it trades AB-1's superior density for universal portability.

5.3 Limitations

- Single-benchmark scope: the Crucible trace is from AB-1's ecosystem, though our 235-trace real dataset partially addresses this.
- No trained embedding: PM-1 tokens have no learned semantics for the model -- they are opaque state identifiers.
- Human-unfriendly: designed for machine-to-machine communication; debugging requires a separate rendering layer.
- Unicode tokenizers not tested: SentencePiece-based models (Gemma, Llama-1/2) tokenize ASCII differently from tiktoken.

5.4 Security Considerations

The 64 KB maximum state size bounds memory allocation for untrusted frames. The Hamming [8,4,4] and parity error detection layers protect against single-bit inference errors in model outputs. However, adversarial inputs designed to exploit the protocol (e.g., injecting command frames into natural-language text) are out of scope. PM-1 assumes a trusted transport between known agent instances.

6. Conclusion

We presented Pro Memoria (PM-1), an ASCII-native binary protocol for token-efficient agent state communication. By encoding 8-bit state as 8-character Morse strings and combining this with a differential state protocol and a two-tier error-correcting lexicon, PM-1 achieves 60-85% token savings versus delta-encoded JSON with zero setup -- no tokenizer extension, no Unicode registration, no configuration changes. The protocol is fully implemented (approximately 750 lines of Python), verified with exhaustive tests (256-byte roundtrip, 128/128 single-error Hamming corrections, 448/448 double-error detections, 56 DSP edge cases), and benchmarked on both synthetic and real agent traces.

PM-1 is not a replacement for AB-1 Braille, which achieves superior density through its tokenizer extension. Rather, PM-1 fills the gap for environments where an extension cannot be installed but token-efficient state communication is still required.

The implementation is open source at github.com/rikirinjani/pro-memoria under Apache-2.0 (code) and CC-BY-4.0 (specification).

References

[1] Tetrahedroned. Agent Braille (AB-1): A Unicode-Based Protocol for Machine-to-Machine State Communication. 2025. github.com/Tetrahedroned/Agent-Braille

[2] Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.

[3] Brown, T. et al. Language Models are Few-Shot Learners. NeurIPS, 2020.

[4] Google. Model Context Protocol (MCP). 2024. github.com/modelcontextprotocol

[5] Google. Agent-to-Agent (A2A) Protocol. 2025. github.com/google/A2A

[6] Sennrich, R. et al. Neural Machine Translation of Rare Words with Subword Units. ACL, 2016.

[7] Hamming, R. W. Error Detecting and Error Correcting Codes. Bell System Technical Journal, 1950.